qc.z(0) and qc.h(0), immediately after qc.h(0) in Line 5, and run the program again: the transpiler reduces all three gates to a single X gate—the identity HZH = X at work.

```
(qiskit_env) # python qiskit15.py
global phase: π/4
q_0 -> 0 ┤ Rz(π/2) ├┤ √X ├┤ Rz(π/2) ├┤M├
                                      └╥┘
    c: 1/═══════════════════════════════╩═
                                        0
fake_marrakesh, qubit 0: SX 36 ns, error 1.3e-04
```

Figure 5: The Hadamard gate translated into native gates for *FakeMarrakesh*

A copying machine that refuses to copy

We end with a short aside on whether a qubit can be copied, a question that the idea of a basis raises naturally: copying the states of a basis turns out to be easy, but what about their superpositions? With its target qubit in $|0\rangle$, a CNOT gate turns $|0\rangle|0\rangle$ into $|0\rangle|0\rangle$ and $|1\rangle|0\rangle$ into $|1\rangle|1\rangle$: it copies the computational basis state of the control qubit onto the target. Have we built a quantum photocopier? Let us feed it $|+\rangle$. A true copier would produce $|+\rangle|+\rangle = (|00\rangle + |01\rangle + |10\rangle + |11\rangle)/2$. Instead, as we have seen ever since the fourth article, the result is $(|00\rangle + |11\rangle)/\sqrt{2}$—the Bell state $|\Phi+\rangle$. We asked for a copy and received entanglement. Quantum gates are linear, so any gate that copies $|0\rangle$ and $|1\rangle$ in this way is forced to turn $|+\rangle|0\rangle$ into $|\Phi+\rangle$ rather than into $|+\rangle|+\rangle$. A similar argument shows that a single device can copy two different states only if they are orthogonal, and even then it fails on their superpositions. This is the no-cloning theorem, published in 1982 by William Wootters and Wojciech Zurek, and independently by Dennis Dieks: no quantum operation can make a perfect copy of an arbitrary unknown state. It does not forbid copying the states of one known orthonormal basis, as the CNOT gate does, nor preparing a known state twice by running its preparation circuit again. We shall return to it, and to its colourful history, with quantum teleportation.

It is now time to conclude. In this article we defined a basis, learned that the hardware measures only in the computational basis, and met the Hadamard gate as the translator that lets us measure in the X basis. We watched amplitudes cancel, saw a mid-circuit measurement spoil the cancellation, buried the myth of trying every answer at once, and followed a Hadamard gate as it was translated into the native gates of a processor model. Finally, a CNOT gate helped us see why an unknown qubit cannot be copied. In the next article, we shall put interference to work on our first quantum algorithm, Deutsch's algorithm. It answers a simple question—is a function of one bit constant or balanced?—with a single query to the function, whereas any classical method needs two queries to be sure. Along the way, we shall meet phase kickback, a trick that hides the answer in a phase until a final Hadamard gate brings it into the open. END

By: Dr Deepu Benson

The author is a free software enthusiast with a keen interest in theoretical computer science. He received his PhD from IIT Palakkad and completed his postdoctoral research at IIT Gandhinagar. He maintains a technical blog.

Continue from page...92

The developer's role

From a developer perspective, supporting observability does not necessarily mean doing extra complicated work. Most of the time it means writing meaningful logs, emitting useful metrics, avoiding silent failures, and adding tracing properly.

AI tools and observability platforms are only as good as the telemetry engineers provide. If the application exposes poor signals, debugging becomes difficult no matter how advanced the tooling is.

In the end, distributed system debugging is about understanding system behaviour during failure conditions. Tools will continue evolving, and AI capabilities will improve rapidly, but engineering judgment and system thinking will remain extremely important for building reliable platforms. END

References

- AWS (2025), AWS DynamoDB Service Disruption Report, Technical incident summary, *https://www.thousandeyes.com/blog/aws-outage-analysis-october-20-2025*
- Azure (2025), Azure Front Door Regional Configuration Analysis, Technical incident review documentation, *https://www.thousandeyes.com/blog/microsoft-azure-front-door-outage-analysis-october-29-2025*
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016), Site Reliability Engineering: How Google Runs Production Systems, O'Reilly Media
- Beyer, B., et al. (2018), The Site Reliability Workbook: Practical Ways to Implement SRE, O'Reilly Media
- Beyer, B., et al. (2020), Building Secure and Reliable Systems: Best Practices for Designing, Implementing, and Maintaining Systems, O'Reilly Media
- Commonwealth Bank (2025), Infrastructure Failure and Service Availability Report, Public incident summary

By: Ajay Lotan Thakur

The author is a Senior IEEE Member, BCS Fellow, and active open source contributor working in cloud-native telecom and distributed infrastructure systems. He is also a Technical Steering Team (TST) member of the Aether Private 5G project. He regularly speaks at engineering events and enjoys simplifying complex distributed systems topics.